

CMPUT275— Bonus Assignment 5 (Winter 2026)

R. Hackman

Due Date (For up to 3.5% bonus marks): Monday April 27th

Per course policy you are allowed to engage in *reasonable* collaboration with your classmates. You must include in the comments of your assignment solutions a list of any students you collaborated with on that particular question.

In this assignment and all following assignments you should test against the sample executables. The sample executables try to help you out and print error messages when receiving invalid input (though they do not catch *all* invalid input). You do not have to replicate any error messages printed, unless the assignment specification specifically asks for error messages. These messages are only in the sample executable to help you catch when you write an invalid test case.

Memory Management: In this assignment you should *NOT* be managing memory manually. That means your solutions should *never* use the `new` or `delete` operators. Instead, you should use STL data types as you see fit.

Compilation Flags: each of your programs should be compiled with at least the flags

```
-std=c++17 -Wall -Werror
```

These are the flags we'll compile your program with, and should they result in a compilation error then your code will not be compiled and ran for testing.

Allowed libraries: `iostream`, `iomanip`, `string`, `sstream`, `vector`, `memory`, `utility`. No other libraries are allowed. You may not need all the aforementioned libraries — use those that you see fit.

1. Maps

A *map* is an ADT whose purpose it is to store a mapping of values from a “key” type K to a “value” type V . It is a common problem in Computer Science that we need to associate values of one type with those of another. As a simple example we may want to associate individuals names with phone numbers to create a phone book. A map would be a useful data type to use to implement such a phone book.

In this question we will create the template class `Map` which has two template parameters, one for the arbitrary key type K and one for the arbitrary value type V . You may assume some things about these types, mainly:

- Type K is orderable (that is, at least `operator!=` and `operator<` will work between K 's).
- Type K can be copy/move initialized and copy/move assigned.
- Type V can be default initialized, as well as copy/move initialized and copy/move assigned.

Your template class `Map` must provide the following operations to the following specifications:

- Default constructor — the default constructor for your `Map` class should construct an empty map
- Big 5 — the big-5 for your class should ensure there are no shallow copies of `lvalue Map` objects and that all resources are cleaned up. **Note:** since you should not be managing memory yourself this should be quite easy!
- `operator[]` — your `Map` type should provide the index operator. The index operator should take a key type K as the operand, and return a reference to the value type object that key maps to. Should the key K not exist in your `Map` then the key K should be added into the map such that it maps to a default constructed value type object (and the reference to that object in your map then returned). For example:

```
Map<string, int> m; // m maps strings to ints, currently empty.
m["foo"] = 5; // since this adds "foo" to the map and returns the reference to the data
// this allows assignment of this key to the value 5!
cout << m["foo"] << endl; // prints 5
```

If the key exists in your `Map` then the index operator should have a worst-case runtime of $\mathcal{O}(\log(n))$. If the key does not exist, thus you must insert it into your map, then your runtime should have a worst-case runtime of $\mathcal{O}(n)$.

Note: typically map data types are implemented with a data type that will typically provide much faster runtimes than those above, but we will not worry about that. If you are interested in reading about that data type you can read online about *hash maps*.

- `operator()` — since a `Map` simply maps values from one type to another (not necessarily distinct) type it can be thought of as the definition of a function! As such, our `Map` type should also provide the function call operator which takes a type K and returns a type V . The difference between this operator and our index operator is that this does *not* return a reference to the value the key maps to (it is simply applying the function, so it should return a value not a reference) *and* if the key does not exist in the map the function call operator will return a default-initialized V object but *should not* add the key to the `Map`!

The function call operator should have a worst-case runtime of $\mathcal{O}(\log(n))$.

Note: It probably makes more sense for the function call operator to *raise an exception*

when the key does not exist. Unfortunately, while exceptions are a core concept to C++ we did not have time to study them. This is another topic you should consider studying if you wish to be a C++ programmer!

- **Iteration** — your `Map` should be a forward-iterable. So, your `Map` must provide method `begin` and `end` which return an iterator type (which implements `operator!=`, prefix `operator++` and unary `operator*`). Your iterator type should return the *keys* of the `Map` when dereferenced, furthermore iteration should take place over the *sorted order* of the keys in the `Map`.

Since your `Map` data type is a template data type it actually defines an infinite family of types. We can of course not test *all* such combinations, so we do just a few simple ones. We have provided test-harness main file which can be compiled to be any map with key/value types of `int`, `str`, or `float` using different preprocessor defines. A provided bash script `buildExecs` can be used to quickly compile all version of the program. You have also been provided with each sample individually, as well as one small basic test case for each of those samples. You should of course write your own further test cases (and test with the different types)!

Deliverables For this question include in your final submission zip your C++ source header file named `myMap.h`. As this is a template type you will write all of your code in the header file.

How to submit: Create a zip file `a5.zip`, make sure that zip file contains your C++ header file `myMap.h`. Assuming `myMap.h` is in your current working directory you can create your zip file with the command

```
$ zip a5.zip myMap.h
```

Upload your file `a5.zip` to the a5 submission link on Canvas.